

LLD Deep Reference

Low Level Design for SDE2 FAANG Interviews

OOP | SOLID | Design Patterns | Full Java Implementations

OOP Fundamentals — 4 Pillars

SOLID Principles — with Java examples

Creational Patterns — Singleton, Factory, Builder, Prototype

Structural Patterns — Adapter, Decorator, Facade, Proxy, Composite

Behavioural Patterns — Strategy, Observer, Command, Template, State, Iterator, Chain

LLD Problem 1: Parking Lot

LLD Problem 2: Rate Limiter

LLD Problem 3: Splitwise (Expense Splitter)

LLD Problem 4: Elevator System

LLD Problem 5: BookMyShow (Movie Ticket Booking)

LLD Problem 6: Snake and Ladder

Interview Protocol + Red Flags + Scoring Rubric

Language: Java | Option A Interview Depth | April 2026

OOP Fundamentals — The 4 Pillars

Know these cold. Interviewers ask "why did you design it this way" — OOP principles are the answer.

1. Encapsulation — Hide internal state, expose behaviour

Private fields, public methods. Validation inside the class. Caller cannot set invalid state.

```
public class BankAccount {

    private double balance; // private -- not directly accessible

    private String accountId;

    public void deposit(double amount) {

        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");

        this.balance += amount; // validation lives here, not in caller

    }

    public void withdraw(double amount) {

        if (amount > balance) throw new InsufficientFundsException();

        this.balance -= amount;

    }

    public double getBalance() { return balance; } // read-only access

}

// Caller cannot set balance = -1000. Must go through validated methods.
```

2. Abstraction — Hide implementation, expose contract

Interface defines WHAT. Implementation defines HOW. Caller only knows the contract.

```
public interface PaymentProcessor {

    boolean processPayment(double amount, String currency);

    void refund(String transactionId);

    PaymentStatus getStatus(String transactionId);

}

// Caller only sees PaymentProcessor -- has no idea if Razorpay or Stripe underneath

public class OrderService {

    private final PaymentProcessor paymentProcessor; // abstraction
```

```

public OrderService(PaymentProcessor processor) {

    this.paymentProcessor = processor;

}

public void placeOrder(Order order) {

    paymentProcessor.processPayment(order.getTotal(), "INR");

}

}

```

3. Inheritance — IS-A relationship, code reuse

Subclass inherits parent behaviour. Override to specialise. Use sparingly — prefer composition.

```

public abstract class Vehicle {

    protected String licensePlate;

    protected int speed;

    public void accelerate(int delta) { speed += delta; }

    public abstract String getType(); // subclass must implement

    public abstract int getCapacity(); // subclass must implement

}

public class Car extends Vehicle {

    @Override public String getType() { return "CAR"; }

    @Override public int getCapacity() { return 4; }

}

public class Truck extends Vehicle {

    private double payloadTons;

    @Override public String getType() { return "TRUCK"; }

    @Override public int getCapacity() { return 2; }

    public double getPayload() { return payloadTons; }

}

```

4. Polymorphism — Same interface, different behaviour

One variable type, many implementations. Swap implementations without changing caller code.

```

List<Vehicle> vehicles = List.of(new Car("KA01"), new Truck("MH02"), new Bike("DL03"));

```

```
// Same method call, different behaviour per type -- polymorphism

for (Vehicle v : vehicles) {

    System.out.println(v.getType() + " capacity: " + v.getCapacity());

}

// Output:

// CAR capacity: 4

// TRUCK capacity: 2

// BIKE capacity: 1

// Caller never changed. Only the list contents changed.
```

Composition vs Inheritance — The Critical Decision

Criterion	Use Inheritance	Use Composition
Relationship	IS-A (Car IS-A Vehicle)	HAS-A (Car HAS-A Engine)
Flexibility	Low — tight coupling to parent	High — swap implementation at runtime
Testability	Hard — must test through parent	Easy — mock the composed object
Code reuse	Direct method reuse	Via delegation to composed object
Rule of thumb	Use rarely, only true IS-A	Default choice (Favour composition over inheritance)

SOLID Principles

These are the "why did you design it this way" answers interviewers want.

S — Single Responsibility Principle

A class should have only ONE reason to change. One class = one job.

```
// BAD: Order class does too many things

public class Order {

    public void calculateTotal() { } // OK -- Order responsibility

    public void saveToDatabase() { } // BAD -- DB is not Order's job

    public void sendConfirmationEmail() { } // BAD -- Email is not Order's job

    public void generateInvoicePDF() { } // BAD -- PDF is not Order's job

}

// GOOD: each class has one reason to change

public class Order { public void calculateTotal() { } }

public class OrderRepository { public void save(Order o) { } }

public class OrderNotificationService { public void sendConfirmation(Order o) { } }

public class InvoiceGenerator { public byte[] generatePDF(Order o) { } }

// Now: changing email logic doesnt touch Order or DB code
```

O — Open/Closed Principle

Open for extension, closed for modification. Add new behaviour without touching existing code.

```
// BAD: adding new payment type requires modifying existing class

public class PaymentService {

    public void process(String type, double amount) {

        if (type.equals("upi")) { processUPI(amount); }

        else if (type.equals("card")) { processCard(amount); } // modify here for each new type

    }

}

// GOOD: extend via new class, never modify existing

public interface PaymentStrategy {

    void process(double amount);

    String getType();

}
```

```

}

public class UpiPayment implements PaymentStrategy {

    public void process(double amount) { /* UPI logic */ }

    public String getType() { return "upi"; }

}

public class NetBankingPayment implements PaymentStrategy { /* add without touching UPI */ }

// Adding new payment = new class only. Zero changes to existing code.

```

L — Liskov Substitution Principle

Subclass must be substitutable for parent without breaking behaviour. If it throws where parent doesn't, it violates LSP.

```

// BAD: Penguin breaks contract of Bird

public class Bird { public void fly() { } }

public class Penguin extends Bird {

    @Override public void fly() {

        throw new UnsupportedOperationException("Penguins cant fly"); // VIOLATES LSP

    }

}

// Code that calls bird.fly() breaks when bird is actually a Penguin

// GOOD: segregate the interface

public interface Bird { String getName(); }

public interface FlyingBird extends Bird { void fly(); }

public class Eagle implements FlyingBird { public void fly() { /* soar */ } }

public class Penguin implements Bird { /* no fly method */ }

// Now: no substitution violation. Penguin never promises to fly.

```

I — Interface Segregation Principle

No client should be forced to implement methods it does not use. Split fat interfaces.

```

// BAD: Robot forced to implement eat() and sleep()

public interface Worker { void work(); void eat(); void sleep(); }

public class Robot implements Worker {

    public void work() { /* ok */ }

    public void eat() { throw new UnsupportedOperationException(); } // robots dont eat

    public void sleep() { throw new UnsupportedOperationException(); } // robots dont sleep

```

```

}

// GOOD: split into focused interfaces

public interface Workable { void work(); }

public interface Eatable { void eat(); }

public interface Sleepable { void sleep(); }

public class Human implements Workable, Eatable, Sleepable { /* implements all */ }

public class Robot implements Workable { /* only work() -- clean */ }

```

D — Dependency Inversion Principle

Depend on abstractions, not concretions. High-level modules should not depend on low-level modules.

```

// BAD: NotificationService tightly coupled to EmailSender

public class NotificationService {

    private EmailSender emailSender = new EmailSender(); // concrete dependency

    // Switching to SMS = rewrite NotificationService

}

// GOOD: depend on abstraction, inject the implementation

public interface MessageSender {

    void send(String recipient, String message);

}

public class NotificationService {

    private final MessageSender sender; // abstraction

    public NotificationService(MessageSender sender) { // injected

        this.sender = sender;

    }

    public void notify(String user, String msg) { sender.send(user, msg); }

}

// Switch from Email to SMS: new SmsSender(), inject it. Zero change to NotificationService.

```

Creational Patterns

How objects are created. Interviewers expect you to name the pattern as you use it.

Singleton — One instance globally

Use when: exactly one instance needed (DB connection, config, thread pool). Thread-safe with double-checked locking.

```
public class DatabaseConnection {

    // volatile prevents JIT reordering of constructor

    private static volatile DatabaseConnection instance;

    private final Connection conn;

    private DatabaseConnection() {

        this.conn = DriverManager.getConnection("jdbc:mysql://...");

    }

    public static DatabaseConnection getInstance() {

        if (instance == null) { // check 1: no lock (fast)

            synchronized (DatabaseConnection.class) {

                if (instance == null) { // check 2: under lock (safe)

                    instance = new DatabaseConnection();

                }

            }

        }

        return instance;

    }

    public Connection getConnection() { return conn; }

}

// Interview Q: why volatile? JIT can reorder new DatabaseConnection() steps.

// Reference can be non-null before object is fully initialised without volatile.
```

Factory Method — Delegate object creation

Use when: caller should not know which concrete class to instantiate. Creation logic centralised.

```
public interface Notification { void send(String message); String getChannel(); }

public class PushNotification implements Notification {
```

```

public void send(String msg) { /* FCM push */ }

public String getChannel() { return "PUSH"; }

}

public class SmsNotification implements Notification {

public void send(String msg) { /* Twilio SMS */ }

public String getChannel() { return "SMS"; }

}

public class EmailNotification implements Notification {

public void send(String msg) { /* SMTP email */ }

public String getChannel() { return "EMAIL"; }

}

// Factory: creation logic in one place

public class NotificationFactory {

public static Notification create(String channel) {

return switch (channel.toUpperCase()) {

case "PUSH" -> new PushNotification();

case "SMS" -> new SmsNotification();

case "EMAIL" -> new EmailNotification();

default -> throw new IllegalArgumentException("Unknown channel: " + channel);

};

}

}

// Usage: Notification n = NotificationFactory.create("SMS");

```

Builder — Complex object construction step by step

Use when: object has many optional parameters. Prevents constructor telescoping (10-arg constructors).

```

public class Order {

private final String userId; // required

private final List<Item> items; // required

private final String couponCode; // optional

private final Address address; // optional

private final String notes; // optional

```

```

private Order(Builder b) { // private -- only Builder can create

this.userId = b.userId; this.items = b.items;

this.couponCode = b.couponCode; this.address = b.address;

this.notes = b.notes;

}

public static class Builder {

private final String userId; // required fields in Builder constructor

private final List<Item> items;

private String couponCode;

private Address address;

private String notes;

public Builder(String userId, List<Item> items) {

if (userId == null || items == null || items.isEmpty())

throw new IllegalArgumentException("userId and items are required");

this.userId = userId; this.items = items;

}

public Builder couponCode(String c) { this.couponCode = c; return this; }

public Builder address(Address a) { this.address = a; return this; }

public Builder notes(String n) { this.notes = n; return this; }

public Order build() { return new Order(this); }

}

}

// Usage: new Order.Builder("u1", items).couponCode("SAVE10").address(addr).build()

```

Prototype — Clone instead of recreate

Use when: creating object is expensive, and you need many similar objects.

```

public abstract class Shape implements Cloneable {

protected String color;

public abstract double area();

public Shape clone() {

try { return (Shape) super.clone(); }

```


Structural Patterns

How classes and objects are composed into larger structures.

Adapter — Make incompatible interfaces work together

Use when: integrating legacy code or third-party libraries with different interfaces.

```
// New system expects this interface

public interface ModernPaymentGateway {

    void pay(String userId, double amount, String currency);

}

// Legacy system we must integrate (cannot modify)

public class LegacyPaymentSystem {

    public boolean makePayment(int customerId, float amount) { return true; }

}

// Adapter: wraps legacy, implements new interface

public class PaymentAdapter implements ModernPaymentGateway {

    private final LegacyPaymentSystem legacy;

    public PaymentAdapter(LegacyPaymentSystem legacy) { this.legacy = legacy; }

    @Override

    public void pay(String userId, double amount, String currency) {

        // Convert types, ignore currency (legacy doesnt support it)

        int customerId = Integer.parseInt(userId);

        boolean success = legacy.makePayment(customerId, (float) amount);

        if (!success) throw new PaymentFailedException();

    }

}
```

Decorator — Add behaviour without modifying original class

Use when: need to add features to objects at runtime, in any combination. Stack decorators.

```
public interface Coffee { double getCost(); String getDescription(); }

public class SimpleCoffee implements Coffee {
```

```

public double getCost() { return 50.0; }

public String getDescription() { return "Coffee"; }

}

// Base decorator

public abstract class CoffeeDecorator implements Coffee {

    protected final Coffee coffee;

    public CoffeeDecorator(Coffee coffee) { this.coffee = coffee; }

}

public class MilkDecorator extends CoffeeDecorator {

    public MilkDecorator(Coffee c) { super(c); }

    public double getCost() { return coffee.getCost() + 15.0; }

    public String getDescription() { return coffee.getDescription() + ", Milk"; }

}

public class SugarDecorator extends CoffeeDecorator {

    public SugarDecorator(Coffee c) { super(c); }

    public double getCost() { return coffee.getCost() + 5.0; }

    public String getDescription() { return coffee.getDescription() + ", Sugar"; }

}

// Usage: stack decorators in any combination

Coffee c = new SugarDecorator(new MilkDecorator(new SimpleCoffee()));

System.out.println(c.getDescription() + " Rs." + c.getCost());

// Output: Coffee, Milk, Sugar Rs.70.0

```

Facade — Simple interface over complex subsystem

Use when: subsystem has many classes. Provide a simple entry point. Hide complexity.

```

// Complex subsystem (many classes, many steps)

class PaymentGateway { boolean charge(String card, double amt) { return true; } }

class InventorySystem { void reserve(String itemId, int qty) { } }

class OrderDatabase { String save(Order o) { return "ORD-123"; } }

class EmailService { void sendConfirmation(String email, String orderId) { } }

```

```

class ShippingSystem { void schedule(String orderId, Address addr) { } }

// Facade: one simple method for the caller

public class OrderFacade {

    private final PaymentGateway payment = new PaymentGateway();

    private final InventorySystem inventory = new InventorySystem();

    private final OrderDatabase db = new OrderDatabase();

    private final EmailService email = new EmailService();

    private final ShippingSystem shipping = new ShippingSystem();

    public String placeOrder(Order order) {

        payment.charge(order.getCard(), order.getTotal());

        inventory.reserve(order.getItemId(), order.getQty());

        String orderId = db.save(order);

        email.sendConfirmation(order.getEmail(), orderId);

        shipping.schedule(orderId, order.getAddress());

        return orderId;

    }

}

// Caller: String id = orderFacade.placeOrder(order); // one line, hides all complexity

```

Proxy — Control access to another object

Use when: need lazy loading, caching, access control, or logging around an object.

```

public interface DataService { List<User> getUsers(); }

public class RealDataService implements DataService {

    public List<User> getUsers() {

        // expensive DB call

        return database.findAllUsers();

    }

}

// Caching proxy -- same interface, adds caching

public class CachingDataServiceProxy implements DataService {

```


Behavioural Patterns

How objects interact and communicate.

Strategy — Swap algorithms at runtime

Use when: multiple algorithms for the same task. Choose at runtime. OCP compliant.

```
public interface SortStrategy { void sort(int[] arr); String getName(); }

public class QuickSort implements SortStrategy {

    public void sort(int[] arr) { /* quicksort impl */ }

    public String getName() { return "QuickSort"; }

}

public class MergeSort implements SortStrategy {

    public void sort(int[] arr) { /* mergesort impl */ }

    public String getName() { return "MergeSort"; }

}

public class Sorter {

    private SortStrategy strategy;

    public Sorter(SortStrategy strategy) { this.strategy = strategy; }

    public void setStrategy(SortStrategy strategy) { this.strategy = strategy; }

    public void sort(int[] arr) {

        System.out.println("Using: " + strategy.getName());

        strategy.sort(arr);

    }

}

// Usage:

Sorter sorter = new Sorter(new QuickSort());

sorter.sort(data);

sorter.setStrategy(new MergeSort()); // swap at runtime

sorter.sort(data);
```

Observer — Event-driven notification (Pub/Sub)

Use when: one object state change should notify many others. Decoupled event system.

```
public interface Observer { void update(String eventType, Object data); }

public class EventBus {

    private final Map<String, List<Observer>> listeners = new HashMap<>();

    public void subscribe(String eventType, Observer observer) {

        listeners.computeIfAbsent(eventType, k -> new ArrayList<>()).add(observer);

    }

    public void unsubscribe(String eventType, Observer observer) {

        listeners.getOrDefault(eventType, List.of()).remove(observer);

    }

    public void publish(String eventType, Object data) {

        listeners.getOrDefault(eventType, List.of())

            .forEach(o -> o.update(eventType, data));

    }

}

// Usage:

EventBus bus = new EventBus();

bus.subscribe("ORDER_PLACED", (type, data) -> emailService.sendConfirmation(data));

bus.subscribe("ORDER_PLACED", (type, data) -> inventoryService.reserve(data));

bus.subscribe("ORDER_PLACED", (type, data) -> analyticsService.track(data));

bus.publish("ORDER_PLACED", order); // notifies all 3 observers
```

Command — Encapsulate request as an object

Use when: need undo/redo, job queues, transaction logging, macro recording.

```
public interface Command { void execute(); void undo(); }

public class TextEditor {

    private StringBuilder text = new StringBuilder();

    public void insertText(String t, int pos) { text.insert(pos, t); }

    public void deleteText(int pos, int len) { text.delete(pos, pos + len); }
```

```

public String getText() { return text.toString(); }

}

public class InsertCommand implements Command {

    private final TextEditor editor;

    private final String text;

    private final int position;

    public InsertCommand(TextEditor e, String t, int pos) {

        this.editor = e; this.text = t; this.position = pos;

    }

    public void execute() { editor.insertText(text, position); }

    public void undo() { editor.deleteText(position, text.length()); }

}

// Command history for undo/redo

public class CommandHistory {

    private final Deque<Command> history = new ArrayDeque<>();

    public void execute(Command cmd) { cmd.execute(); history.push(cmd); }

    public void undo() { if (!history.isEmpty()) history.pop().undo(); }

}

```

Template Method — Skeleton in parent, steps in subclass

Use when: algorithm structure is fixed but specific steps vary. Prevents code duplication.

```

public abstract class DataProcessor {

    // Template method: defines the skeleton, cannot be overridden

    public final void process() {

        readData(); // step 1: subclass implements

        validateData(); // step 2: has default, subclass can override

        processData(); // step 3: subclass must implement

        writeData(); // step 4: subclass implements

        generateReport(); // step 5: default impl, can override

    }

}

```

```

protected abstract void readData();

protected abstract void processData();

protected abstract void writeData();


protected void validateData() { System.out.println("Default validation"); }

protected void generateReport() { System.out.println("Default report"); }

}


public class CSVProcessor extends DataProcessor {

protected void readData() { /* read CSV */ }

protected void processData() { /* parse CSV rows */ }

protected void writeData() { /* write to DB */ }

@Override

protected void generateReport() { /* CSV-specific report */ }

}

```

Chain of Responsibility — Pass request along handler chain

Use when: multiple handlers may process a request. Priority ordering. Each handler decides to handle or pass.

```

public abstract class SupportHandler {

protected SupportHandler next;


public SupportHandler setNext(SupportHandler next) {

this.next = next; return next; // fluent for chaining

}


public abstract void handle(Ticket ticket);


protected void passToNext(Ticket ticket) {

if (next != null) next.handle(ticket);

else System.out.println("Ticket unresolved: " + ticket.getId());

}

}


public class L1Support extends SupportHandler {

public void handle(Ticket t) {

```


LLD Problem 1: Parking Lot

Most frequently asked LLD problem. Expected at Amazon, Flipkart, Microsoft, Grab.

Requirements (ask these)

Question	Assumption
Multiple floors?	Yes — configurable
Vehicle types?	Bike, Car, Truck
Pricing?	Per hour, different rates per type
Multiple entry/exit?	Yes
Display available spots?	Yes per floor and type
Concurrent access?	Yes — thread safe

Entities Identified

ParkingLot (Singleton), Floor, ParkingSpot, Vehicle hierarchy, Ticket, PricingStrategy, EntryGate, ExitGate

```
// ■■ Enums ████████████████████████████████████████████████████████████████████████
```

```
public enum VehicleType { BIKE, CAR, TRUCK }
```

```
public enum SpotType { SMALL, MEDIUM, LARGE }
```

```
public enum SpotStatus { AVAILABLE, OCCUPIED }
```

```
public enum TicketStatus { ACTIVE, COMPLETED }
```

```
// ■■ Vehicle Hierarchy ████████████████████████████████████████████████████████████████████████
```

```
public abstract class Vehicle {
```

```
    protected final String licensePlate;
```

```
    protected final VehicleType type;
```

```
    public Vehicle(String plate, VehicleType type) {
```

```
        this.licensePlate = plate; this.type = type;
```

```
    }
```

```
    public VehicleType getType() { return type; }
```

```
    public String getLicensePlate() { return licensePlate; }
```

```
}
```

```
public class Bike extends Vehicle { public Bike(String p) { super(p, VehicleType.BIKE); } }
```

[illegible]


```

    activeTickets.put(ticket.getTicketId(), ticket);

    return ticket;
}

public double exit(String ticketId) {
    Ticket ticket = activeTickets.remove(ticketId);

    if (ticket == null) throw new IllegalArgumentException("Invalid ticket: " + ticketId);

    ticket.markExit();

    ticket.getSpot().vacate();

    return pricing.calculate(ticket.getDurationHours(), ticket.getVehicle().getType());
}

public Map<SpotType, Long> getAvailability(int floorNumber) {
    Floor floor = floors.get(floorNumber);

    return Map.of(
        SpotType.SMALL, floor.getAvailableCount(SpotType.SMALL),
        SpotType.MEDIUM, floor.getAvailableCount(SpotType.MEDIUM),
        SpotType.LARGE, floor.getAvailableCount(SpotType.LARGE)
    );
}
}

```

Patterns Used — Say These Out Loud

- + Singleton: ParkingLot — exactly one instance of the lot
- + Strategy: PricingStrategy — swap pricing algorithm without touching ParkingLot
- + Factory: can wrap Vehicle creation in VehicleFactory
- + ConcurrentHashMap for thread-safe ticket tracking
- + synchronized on ParkingSpot methods — thread-safe spot reservation

LLD Problem 2: Rate Limiter

Asked at Rubrik, Rippling, Amazon. Tests Strategy pattern + thread safety + extensibility.

Requirements

Question	Assumption
Multiple algorithms?	Yes — Token Bucket, Sliding Window, Fixed Window
Rate limit by what?	User ID, IP address, API key, endpoint — configurable
Distributed?	Single node for LLD, mention Redis for distributed
Thread safe?	Yes — concurrent requests
Response headers?	Yes — X-RateLimit-Remaining, X-RateLimit-Reset

[illegible]

[illegible]

LLD Problem 3: Splitwise (Expense Splitter)

Tests entity design, algorithm (settlement), and extensibility. Asked at Flipkart, Razorpay.

Requirements

Question	Assumption
Split types?	Equal, Exact amount, Percentage, Shares
Multiple currencies?	Single currency for simplicity
Settlement?	Minimum transactions to settle all balances
Groups?	Yes — users can be in multiple groups
History?	Yes — list expenses and balances


```

public class Group {

    private final String groupId;

    private final String name;

    private final List<User> members;

    private final List<Expense> expenses = new ArrayList<>();

    private final BalanceManager balanceManager = new BalanceManager();

    public Group(String name, List<User> members) {

        this.groupId = UUID.randomUUID().toString();

        this.name = name; this.members = new ArrayList<>(members);

    }

    public void addExpense(Expense expense) {

        expenses.add(expense);

        String payerId = expense.getPaidBy().getId();

        for (Split split : expense.getSplits()) {

            String debtorId = split.getUser().getId();

            if (!debtorId.equals(payerId)) {

                // debtor owes payer this split amount

                balanceManager.updateBalance(debtorId, payerId, split.getAmount());

            }

        }

    }

    public List<String> settle() {

        return balanceManager.settleAll(members);

    }

    public double getBalance(String fromUserId, String toUserId) {

        return balanceManager.getBalance(fromUserId, toUserId);

    }

}

```

- + Strategy: Split hierarchy — Equal/Exact/Percentage/Shares as separate classes
- + Template method implied in Split.calculateAmount abstract method
- + Greedy algorithm for minimum transactions settlement

LLD Problem 4: Elevator System

Tests state machines, scheduling algorithms, and thread safety. Asked at Google India.

Requirements

Question	Assumption
How many elevators?	N elevators (configurable)
How many floors?	M floors (configurable)
Scheduling algo?	SCAN (elevator algorithm) — go in one direction, serve all requests, reverse
Direction buttons?	Up and down buttons on each floor
Concurrent requests?	Yes — thread safe

[illegible]


```
}  
}  
  
// ■■ Dispatcher ████████████████████████████████████████████████████████████████████████████  
  
public class ElevatorDispatcher {  
  
    private final List<Elevator> elevators;  
  
    public ElevatorDispatcher(List<Elevator> elevators) {  
  
        this.elevators = elevators;  
  
    }  
  
    // Assign request to best elevator (lowest score)  
  
    public Elevator dispatch(ElevatorRequest request) {  
  
        return elevators.stream()  
  
            .filter(e -> e.getStatus() != ElevatorStatus.MAINTENANCE)  
  
            .min(Comparator.comparingInt(e -> e.calculateScore(request)))  
  
            .orElseThrow(() -> new RuntimeException("No elevators available"));  
  
    }  
  
}
```

```
// ■■ Building (Facade) ████████████████████████████████████████████████████████████████████████████  
  
public class Building {  
  
    private final List<Elevator> elevators;  
  
    private final ElevatorDispatcher dispatcher;  
  
    private final int totalFloors;  
  
    public Building(int numElevators, int numFloors) {  
  
        this.totalFloors = numFloors;  
  
        this.elevators = new ArrayList<>();  
  
        for (int i = 0; i < numElevators; i++) {  
  
            elevators.add(new Elevator(i, 0, numFloors)); // all start at ground  
  
        }  
  
        this.dispatcher = new ElevatorDispatcher(elevators);  
  
    }  
  
}
```

```

public void requestElevator(int floor, Direction direction) {

    if (floor < 0 || floor >= totalFloors)

        throw new IllegalArgumentException("Invalid floor: " + floor);

    ElevatorRequest request = new ElevatorRequest(floor, direction);

    Elevator assigned = dispatcher.dispatch(request);

    assigned.addRequest(floor);

    System.out.println("Elevator " + assigned.getElevatorId() + " assigned to floor " + floor);

}

public void pressFloorButton(int elevatorId, int destinationFloor) {

    Elevator e = elevators.get(elevatorId);

    e.addRequest(destinationFloor);

}

public void step() {

    elevators.forEach(e -> {

        int next = e.getNextFloor();

        if (next != e.getCurrentFloor()) e.moveToFloor(next);

    });

}

}

```

- + State machine: Direction and ElevatorStatus enums track elevator state clearly
- + SCAN algorithm: TreeSet for ordered floor requests in each direction
- + Strategy: calculateScore can be swapped for different dispatching algorithms
- + Facade: Building hides complexity of dispatcher + multiple elevators
- + synchronized on Elevator methods — thread safe floor requests

LLD Problem 5: BookMyShow (Movie Ticket Booking)

Requirements

Question	Assumption
Concurrent bookings?	Yes — two users cannot book same seat
Multiple movies/shows?	Yes
Seat types?	Regular, Premium, VIP with different pricing
Payment?	Simplified — just reserve then confirm
Cancellation?	Yes — release seat back
Expiry?	Reservation expires in 10 minutes if unpaid

```
// Enum Enums
public enum SeatType { REGULAR, PREMIUM, VIP }

public enum SeatStatus { AVAILABLE, RESERVED, BOOKED }

public enum BookingStatus { PENDING, CONFIRMED, CANCELLED, EXPIRED }

// Seat
public class Seat {

    private final String seatId; // e.g. "A1", "B12"

    private final int row;

    private final int number;

    private final SeatType type;

    private volatile SeatStatus status = SeatStatus.AVAILABLE;

    private volatile String reservedByBookingId;

    private volatile LocalDateTime reservedAt;

    private static final int RESERVATION_TIMEOUT_MINUTES = 10;

    public Seat(String id, int row, int num, SeatType type) {

        this.seatId = id; this.row = row;

        this.number = num; this.type = type;

    }

    // synchronized: only one thread can reserve at a time
```


[illegible]


```

.allMatch(s -> s.confirm(bookingId));

if (!allConfirmed) {

    booking.expire(); // some seats expired, cancel booking

    throw new BookingExpiredException("Reservation expired");

}

booking.confirm();

}

public void cancelBooking(String bookingId) {

    Booking booking = getBooking(bookingId);

    booking.cancel();

}

private Booking getBooking(String id) {

    Booking b = bookings.get(id);

    if (b == null) throw new IllegalArgumentException("Booking not found: " + id);

    return b;

}

}

```

- + synchronized on Seat.reserve() — prevents two users booking same seat
- + Reservation timeout — auto-release if payment not completed in 10 min
- + Atomic-or-rollback pattern in createBooking — no partial reservations
- + State machine: BookingStatus transitions are irreversible (CONFIRMED cannot go back to PENDING)

LLD Problem 6: Snake and Ladder

Requirements

Question	Assumption
Board size?	Configurable (default 10x10 = 100 squares)
Players?	2-4 players
Snakes and ladders?	Configurable count and positions
Dice?	One dice (1-6), configurable sides
Win condition?	First to exactly reach 100 wins

[illegible]


```

    } else {

        current.moveTo(newPosition);

    }

    if (current.getCurrentPosition() == board.getSize()) {

        winner = current;

        System.out.println(current.getName() + " WINS!");

        return winner;

    }

    playerQueue.offer(current); // re-queue for next turn

    return null; // game continues

}

public void playGame() {

    while (!isGameOver()) { playTurn(); }

}

public boolean isGameOver() { return winner != null; }

public Player getWinner() { return winner; }

}

```

- + Strategy: Dice configurable sides — can swap for loaded dice in tests
- + Template method implied: `Cell.getEndPosition()` abstract — Snake and Ladder override
- + Validation in Board constructor — fail fast on invalid positions
- + `Queue<Player>` for circular turn management — clean data structure choice
- + Bust rule handled cleanly — `newPosition == -1` means stay

LLD Interview Protocol + Red Flags + Scoring

How to structure your 45 minutes and what interviewers are actually grading.

The 45-Minute Framework

Time	What to Do	Common Mistake
0-3 min	Clarify requirements. Scale? Concurrency? Which features in scope?	Starting to code immediately without clarifying
3-6 min	Identify entities and relationships. Say them out loud.	Jumping to code before knowing your classes
6-10 min	Define interfaces and enums FIRST. No implementation yet.	Writing implementation before contract
10-30 min	Implement core classes. Call out patterns as you use them.	Silently coding without explaining decisions
30-35 min	Thread safety, edge cases, extensibility discussion.	Ending without showing you thought about failures
35-40 min	Follow-up: "how would you make this distributed/testable?"	Stopping when asked this — engage with it

The Magic Sentence — Say This Every Time

```
"I see these main entities: [list them]"

"I'm going to define interfaces first, then implement"

"I'm using [Pattern] here because [reason]"

"I'm making this synchronized because [concurrent access reason]"

"If we needed to extend this to support [X], we would [Y]"
```

Design Patterns — When to Use Which

Pattern	Use When You See...	Signal to Interviewer
Singleton	One global instance (Parking Lot, Config, Thread Pool)	You know when global state is appropriate
Factory	Caller should not know concrete class. Creation logic complex.	You separate creation from usage
Builder	Many optional parameters. Complex object setup.	You prevent telescoping constructors
Strategy	Multiple algorithms for same task, swappable at runtime.	You know OCP — add without modifying
Observer	One event, many listeners. Decoupled notification.	You know event-driven design

Decorator	Add features without modifying original. Stack features.	You prefer composition over inheritance
Facade	Complex subsystem, need simple entry point.	You can hide complexity
Command	Undo/redo, queuing operations, transaction log.	You think about operation history
Template Method	Fixed algorithm skeleton, varying steps.	You eliminate duplicate code across subclasses
Chain of Responsibility	Request may be handled by one of many handlers.	You know priority-based processing

Thread Safety Patterns

```

Immutable objects: make fields final, no setters -> inherently thread safe

synchronized method: simplest. Fine-grained is better (sync blocks).

ConcurrentHashMap: thread-safe map. Use instead of HashMap for shared state.

AtomicInteger/AtomicReference: lock-free single-variable operations.

volatile: visibility guarantee for simple flags. Not for compound actions.

ReentrantLock: when you need tryLock() or lockInterruptibly()

Rule: if shared state is modified by multiple threads -> you MUST address thread safety

Even in LLD: say "this needs to be synchronized because..." shows SDE2 maturity

```

Red Flags — What Interviewers Penalise

- Jumping to code before identifying entities — shows no design thought
- God class — one class with 500 lines doing everything
- if/else chains where polymorphism should be used
- Making everything public — no encapsulation
- Static methods everywhere — untestable, not OOP
- Not using interfaces — hard-coding concrete classes
- Forgetting thread safety for shared state
- Using design pattern name without explaining why
- Cannot extend your own design — "id have to rewrite to add X"
- No input validation — not thinking about edge cases

Green Flags — What Gets You Hired

- + Start with entities and relationships before any code
- + Define interfaces before implementations — contract first
- + Name patterns as you use them with justification
- + Add synchronized where shared state is modified
- + Validate inputs in constructors — fail fast

- + Show extensibility: "to add X we only need a new class, no existing code changes"
- + Make key fields final — show immutability awareness
- + Use Optional instead of null returns
- + Discuss trade-offs: "I used composition here instead of inheritance because..."

Scoring Rubric — What Interviewers Grade

Area	Weight	What They Check
OOP Correctness	25%	Encapsulation, proper use of inheritance, polymorphism, abstraction
Design Patterns	25%	Right pattern for right problem. Named and justified.
Extensibility	20%	Can you add new features without rewriting existing code? OCP.
Thread Safety	15%	Identified shared state, used appropriate synchronisation
Code Quality	15%	Readable names, input validation, exception handling, no god classes

Company-Specific LLD Expectations

Company	LLD Bar	What They Focus On
Amazon	Medium. 45 min. One full problem.	Working code + extensibility. LP: "design something you'd be proud of"
Flipkart	High. Dedicated LLD round.	Complete implementation. Thread safety. Error handling.
Google	Lower LLD bar. More design discussion.	Testability. "How would you unit test this?" Clean interfaces.
Grab/Gojek	Medium. May combine HLD+LLD.	Practical patterns. Real-world scenarios. Concurrency awareness.
Rubrik	High. May add distributed extension.	"Make your parking lot work across 3 DCs." Complexity handling.
Rippling	High. Speed matters.	Complete solution fast. Pattern naming. Extensibility.

Quick Reference — Patterns by Problem

Problem	Key Patterns	Thread Safety
Parking Lot	Singleton, Strategy (pricing), Factory (vehicles)	synchronized on ParkingSpot.park/vacate
Rate Limiter	Strategy (algorithm), Factory, Facade	synchronized on tryAcquire
Splitwise	Template Method (Split types), Facade (Group)	Not critical for LLD scope
Elevator	Strategy (dispatch), State machine (Direction), Facade	synchronized on Elevator.addRequest
BookMyShow	State machine (Seat/Booking), Factory	synchronized on Seat.reserve/confirm

